System Setup and Lab Configuration Report

Prepared by: Aicha Nait Dayaa

Date: February 7, 2026

1 Chapter 1: Setup

In this chapter, we set up the environment by downloading VMware, Vagrant, and the necessary plugins. This will include configuring a virtual machine using Vagrant.

```
mkdir -p ~/dev/labs/vm

cd ~/dev/labs/vm

curl -o Vagrantfile https://gist.githubusercontent.com/Bahlaouane-Hamza/84
    a7a9088c6a1af2d8806fbe2692deb2/raw/
    f883a0fe92597d551de90e7da7b04bbfea6d1e28/Vagrantfile

vagrant up --provider vmware_desktop
```

The script will create a directory, download a pre-configured `Vagrantfile`, and start the virtual machine using VMware as the provider.

2 Chapter 2: SSH Setup

This section describes how to set up SSH access to the virtual machine and configure passwordless authentication.

2.1 Part 1: Initial SSH Connection

To configure SSH, first, we generate the SSH configuration for Vagrant:

```
vagrant ssh-config
```

Figure 1: SSH configuration details

We create the user `alpha` on the host `127.0.0.1` and set its password to `123456` using the following commands:

```
sudo useradd -m -s /bin/bash alpha
sudo passwd alpha
```

We connect to the VM using password authentication:

```
ssh alpha@127.0.0.1 -p 2222
```

2.2 Part 2: Implementing Passwordless SSH

2.2.1 Generating SSH Key Pair

On my local machine, I generate a new SSH key pair:

```
ssh-keygen -t rsa -b 4096 -C "a.naitdayaa@void.fr"
```

During the generation, we:

- Accept or write the new location (`~/.ssh/id_rsa_alpha`).
- Set the passphrase for extra security.

The two files generated:

- `id_rsa_alpha`: Private key.
- `id_rsa_alpha.pub`: Public key.

2.2.2 Copying the Public Key to the VM

We use `scp` to transfer the public key to the VM:

```
scp ~/.ssh/id_rsa_alpha.pub alpha@127.0.0.1:/tmp/
```

Once logged into the VM, we configure the key:

```
ssh alpha@127.0.0.1 -p 2222
# Inside the VM:
mkdir -p ~/.ssh
cat ~/tmp/id_rsa_alpha.pub >> ~/.ssh/authorized_keys
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
rm ~/id_rsa_alpha.pub
exit
```

We set the correct directory and file permissions:

- `chmod 700 ~/.ssh`: Owner-only access to the directory.
- `chmod 600 authorized_keys`: Owner read/write only.

2.2.3 Adding the Key to SSH Agent

We add the private key to the SSH agent:

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_rsa_alpha
```

2.3 Part 3: The `ssh-copy-id` Shortcut

Instead of manually copying the key, we use the `ssh-copy-id` utility:

```
ssh-copy-id -p [PORT] alpha@127.0.0.1
```

This single command does everything we did manually, including:

- Copies the public key to the server.
- Adds it to `authorized_keys`.
- Sets the correct permissions.

3 Chapter 3: SSL/TLS Testing

In this section, we analyze SSL/TLS errors and ways to fix them.

3.1 Part 1: Testing an Expired Certificate

We tested how `curl` handles an expired certificate:

```
curl -v https://expired.badssl.com/
```

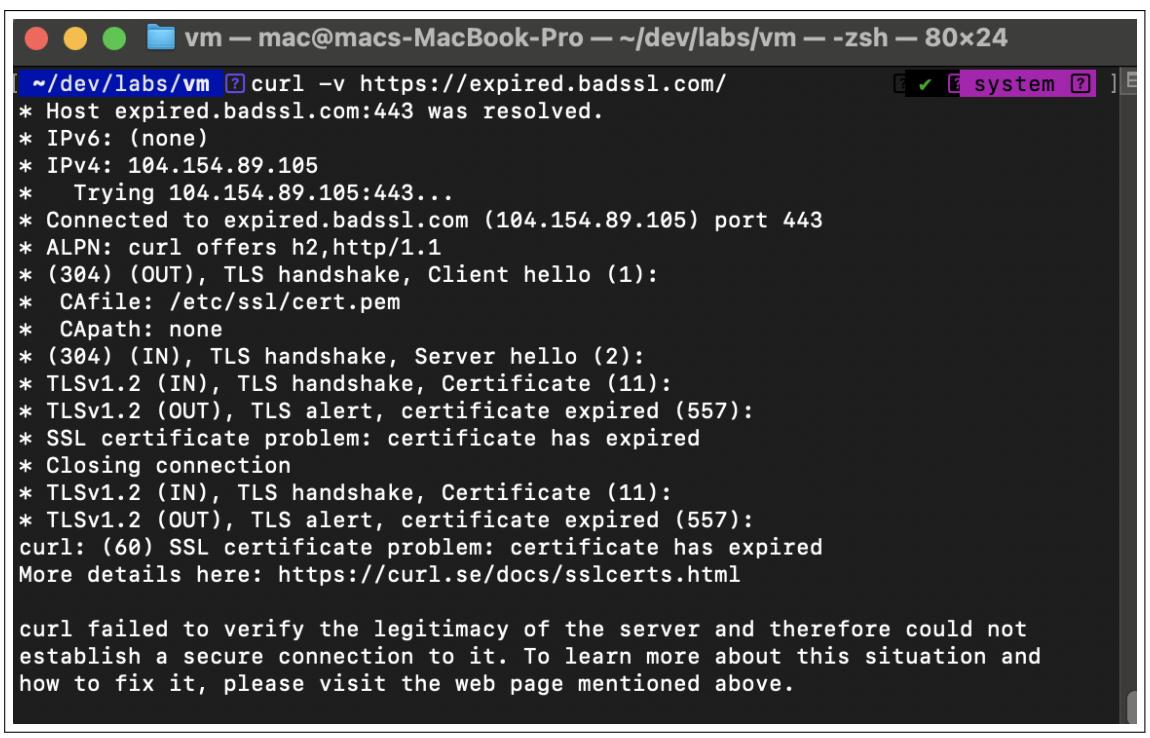


Figure 2: curl rejecting an expired SSL certificate

This error occurs because the certificate's validity period has passed.

3.2 Part 2: Testing a Self-Signed Certificate

We tested a self-signed certificate:

```
curl -v https://self-signed.badssl.com/
```

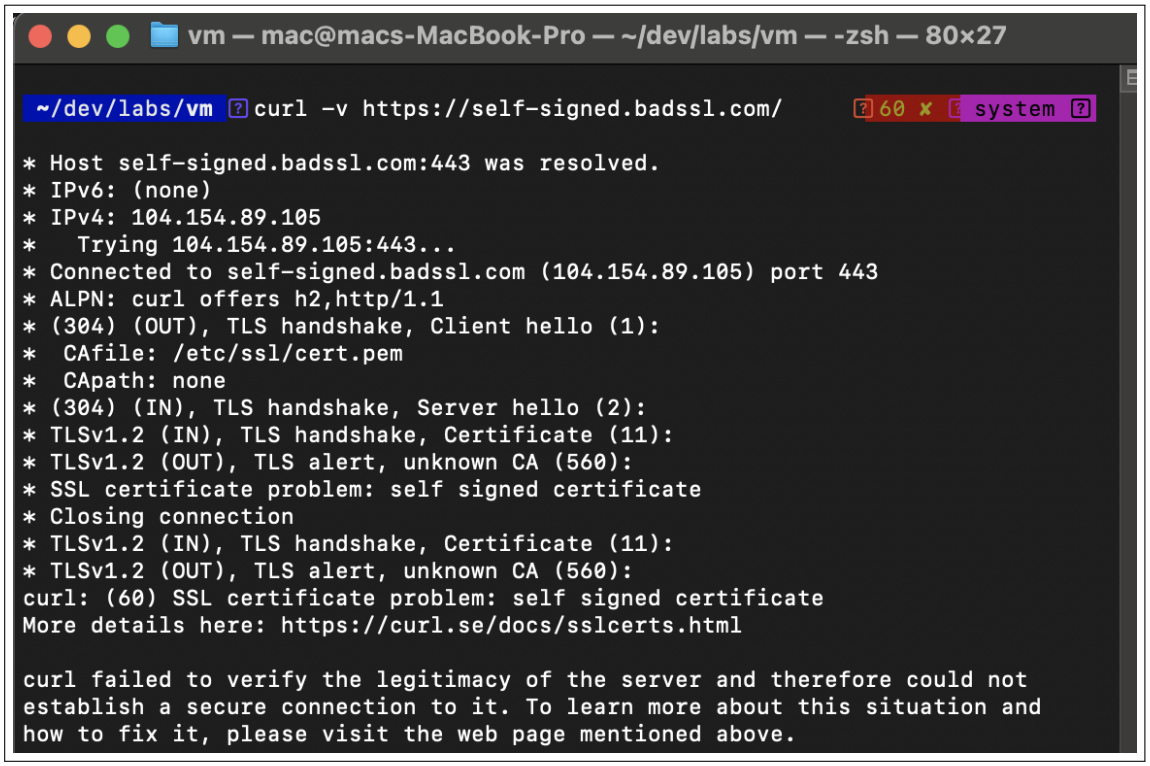


Figure 3: curl rejecting a self-signed certificate

A self-signed certificate is not trusted by default because it is not signed by a trusted Certificate Authority (CA).

3.3 Part 3: Analyzing SSL Errors

Error Type	What It Means	Why It's a Problem
Expired Certificate	The certificate was valid once but has passed its expiration date	The website hasn't renewed their cert - could be abandoned or compromised
Self-Signed Certificate	No trusted third party verified the website's identity	Anyone could claim to be this website - no way to verify it's legitimate

Table 1: Comparison of SSL certificate errors

3.4 Part 4: Deep Dive with OpenSSL

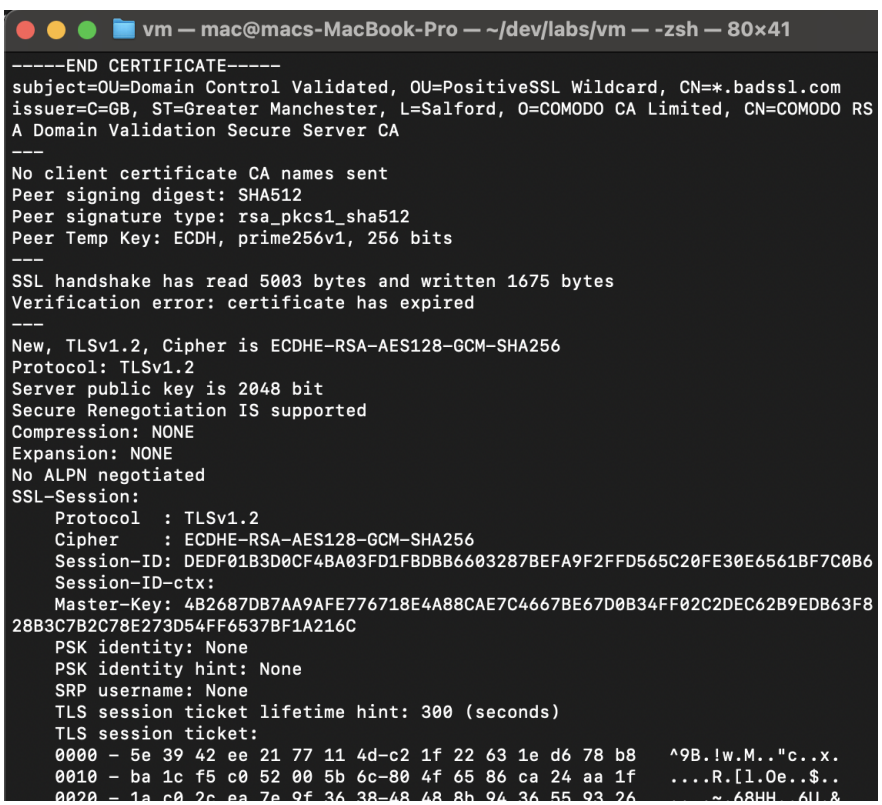
We use OpenSSL to inspect the certificate details:

```
openssl s_client -connect expired.badssl.com:443 </dev/null
```

```
~/dev/labs/vm openssl s_client -connect expired.badssl.com:443

Connecting to 104.154.89.105
CONNECTED(00000005)
depth=2 C=GB, ST=Greater Manchester, L=Salford, O=COMODO CA Limited, CN=COMODO R
SA Certification Authority
verify return:1
depth=1 C=GB, ST=Greater Manchester, L=Salford, O=COMODO CA Limited, CN=COMODO R
SA Domain Validation Secure Server CA
verify return:1
depth=0 OU=Domain Control Validated, OU=PositiveSSL Wildcard, CN=*.badssl.com
verify error:num=10:certificate has expired
notAfter=Apr 12 23:59:59 2015 GMT
verify return:1
depth=0 OU=Domain Control Validated, OU=PositiveSSL Wildcard, CN=*.badssl.com
notAfter=Apr 12 23:59:59 2015 GMT
verify return:1
---
Certificate chain
 0 s:OU=Domain Control Validated, OU=PositiveSSL Wildcard, CN=*.badssl.com
```

Figure 4: OpenSSL showing detailed certificate information



```
vm — mac@macs-MacBook-Pro — ~/dev/labs/vm — -zsh — 80×41
-----END CERTIFICATE-----
subject=OU=Domain Control Validated, OU=PositiveSSL Wildcard, CN=*.badssl.com
issuer=C=GB, ST=Greater Manchester, L=Salford, O=COMODO CA Limited, CN=COMODO RS
A Domain Validation Secure Server CA
---
No client certificate CA names sent
Peer signing digest: SHA512
Peer signature type: rsa_pkcs1_sha512
Peer Temp Key: ECDH, prime256v1, 256 bits
---
SSL handshake has read 5003 bytes and written 1675 bytes
Verification error: certificate has expired
---
New, TLSv1.2, Cipher is ECDHE-RSA-AES128-GCM-SHA256
Protocol: TLSv1.2
Server public key is 2048 bit
Secure Renegotiation IS supported
Compression: NONE
Expansion: NONE
No ALPN negotiated
SSL-Session:
    Protocol  : TLSv1.2
    Cipher    : ECDHE-RSA-AES128-GCM-SHA256
    Session-ID: DEDF01B3D0CF4BA03FD1FBDBB6603287BEFA9F2FFD565C20FE30E6561BF7C0B6
    Session-ID-ctx:
    Master-Key: 4B2687DB7AA9AFE776718E4A88CAE7C4667BE67D0B34FF02C2DEC62B9EDB63F8
28B3C7B2C78E273D54FF6537BF1A216C
    PSK identity: None
    PSK identity hint: None
    SRP username: None
    TLS session ticket lifetime hint: 300 (seconds)
    TLS session ticket:
    0000 - 5e 39 42 ee 21 77 11 4d-c2 1f 22 63 1e d6 78 b8   ^9B.!w.M.."c..x.
    0010 - ba 1c f5 c0 52 00 5b 6c-80 4f 65 86 ca 24 aa 1f   ....R.[l.Oe..$..
```

Figure 5: OpenSSL showing detailed certificate information

3.5 Part 5: Solutions to SSL Problems

Solutions to SSL Problems

For Expired Certificates:

1. Contact the Certificate Authority (CA) to renew the certificate.
2. Use `certbot` for auto-renewal:

```
sudo certbot renew
```

3. Replace the old certificate files on the server.
4. Restart the web server to load the new certificate.

For Self-Signed Certificates:

1. Obtain a certificate from a trusted CA like Let's Encrypt.
2. Install the CA-signed certificate on the server.
3. Ensure the full certificate chain is installed.

4 Chapter 4: Cronjob Setup

In this chapter, we discuss how to schedule a cronjob to delete files modified within the last day.

4.1 Creating Temporary Files

First, we create the directory and files:

```
mkdir ~/temp
touch ~/temp/file{1..10}.txt
```

Next, we create the script `clean_temp.sh`:

```
#!/bin/bash
find ~/temp -type f -mtime -1 -delete -print
```

4.2 Scheduling the Cronjob

To schedule the cronjob, we add the following line to `crontab`:

```
* * * * * /home/alpha/clean_temp.sh >> /home/alpha/cronjob.log 2>&1
```

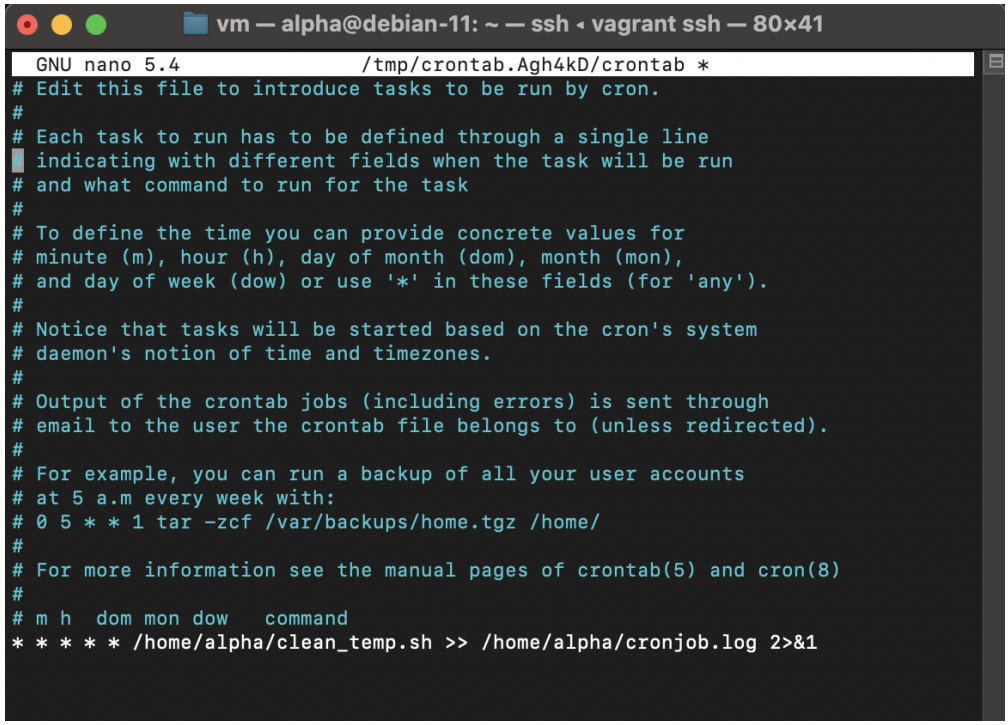


Figure 6: Cronjob Setup

4.3 Verifying Cronjob Execution

To verify the cronjob, we check the cron logs or the logs in the file cronjob.log created:

```
tail -f /var/log/syslog
```

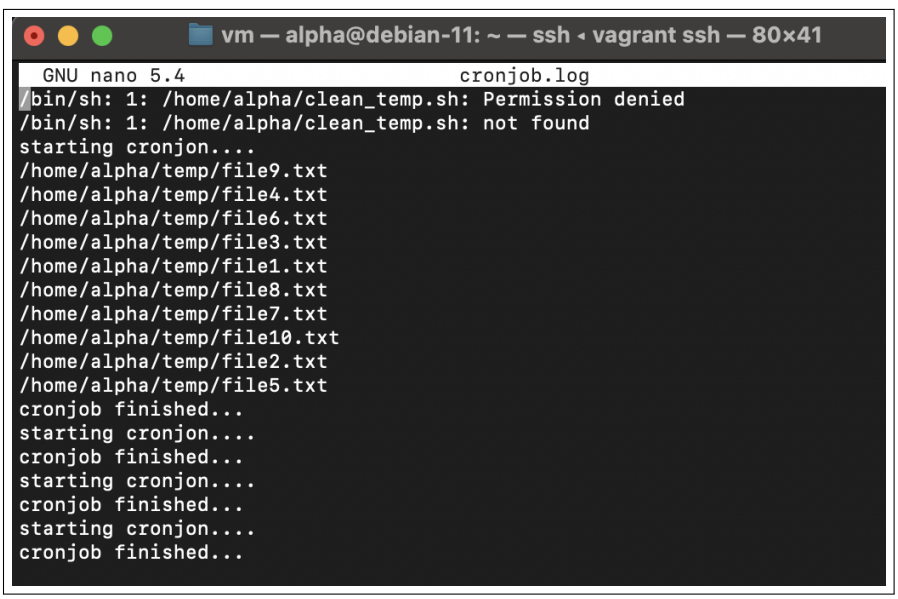


Figure 7: Cronjob Logs

5 Chapter 5: Permissions and Webserver Installation

In this section, we explore permissions management and web server installation.

5.1 Configuring Granular `sudo` Access

We need to grant limited `sudo` access to the user `alpha` for the `ifconfig` command only. This can be done by modifying the `sudoers` file through the command visudo.

We should NEVER edit `/etc/sudoers` directly with a regular text editor. Hence the use `visudo`:

```
sudo visudo
```

Why `visudo`?

- It locks the file so only one person can edit at a time
- It validates syntax before saving
- If there's an error, it won't save and lock me out of sudo

We need to add the following line (23) to allow `alpha` to run `ifconfig` with `sudo`:

```
alpha ALL=(ALL) NOPASSWD: /sbin/ifconfig
```

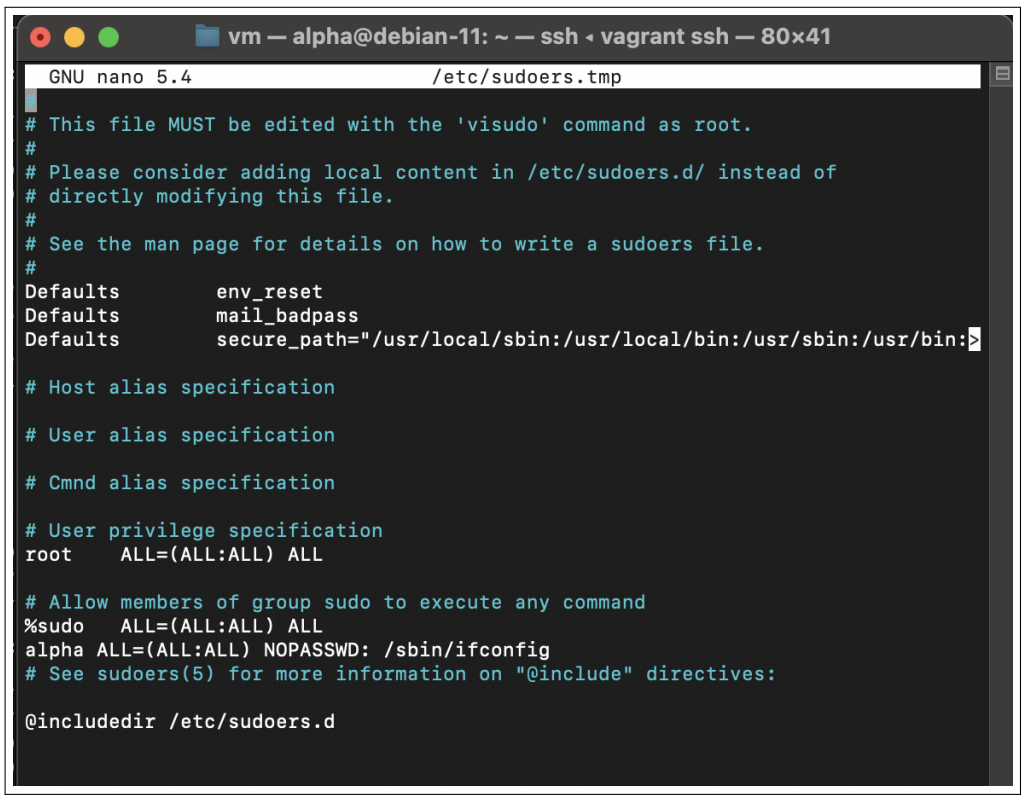


Figure 8: Editing the sudoers file with visudo

We verify by running the `ifconfig` command as `alpha`.

```
alpha@debian-11:~$ ifconfig
-bash: ifconfig: command not found
alpha@debian-11:~$ sudo ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 172.16.203.130  netmask 255.255.255.0  broadcast 172.16.203.255
        inet6 fe80::20c:29ff:fe85:cf14  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:85:cf:14  txqueuelen 1000  (Ethernet)
        RX packets 18787  bytes 14197250 (13.5 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 7887  bytes 896304 (875.2 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
        device interrupt 18  memory 0xfd4a0000-fd4c0000

eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.33.10  netmask 255.255.255.0  broadcast 192.168.33.255
        inet6 fe80::20c:29ff:fe85:cf1e  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:85:cf:1e  txqueuelen 1000  (Ethernet)
        RX packets 479  bytes 52416 (51.1 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 152  bytes 17693 (17.2 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
        device interrupt 16  memory 0xfd2a0000-fd2c0000

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 94  bytes 43497 (42.4 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 94  bytes 43497 (42.4 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Figure 9: Editing the sudoers file with visudo

5.2 The Permission Denied Problem

In an attempted to download a file directly to `/opt/`:

```
curl -o /opt/bat.zip https://github.com/sharkdp/bat/archive/refs/tags/v0
    .24.0.zip
```

We get an error:

```
curl: (23) Failed to create the file /opt/bat.zip: Permission denied
    Failure writing output to destination
```

We then need to examine the directory permissions:

```
ls -ld /opt/
```

Output:

```
drwxr-xr-x 2 root root 4096 Feb 6 10:00 /opt/
```

- `rwx`: Owner (root) can read, write, execute
- `r-x`: Group (root) can read and execute, but not write

- `r-x`: Others can read and execute, but not write
- `root root`: Owner is root, group is root

Since I am the user "alpha" (not root), I fell into the "others" category, which only had read and execute permissions - no write access! The requirement was to get the file into `/opt/` without using `sudo` with `curl`. Instead of downloading the file to a directory I control and then moving it with `sudo`, we can solve this by changing the ownership of the `/opt/` directory (or the destination file) to the `alpha` user. This will allow us to write directly to that directory without needing root permissions. The steps to do this are as follows:

```
# 1. Change the ownership of the /opt/ directory or specific file

sudo chown alpha:alpha /opt/

# 2. Download the file directly to /opt/

curl -o /opt/bat.zip https://github.com/sharkdp/bat/archive/refs/tags/v0
    .24.0.zip
```

This allows the user `alpha` to download the file directly into `/opt/`, bypassing the need for `sudo` during the `curl` command.

5.3 Extracting Without unzip

We needed to extract the ZIP file, but without using `unzip`.

5.3.1 Using Python to Extract

Python has built-in ZIP support:

```
python3 -m zipfile -e /opt/bat.zip /opt/
```

Then we verify the extraction:

```
ls -la /opt/
```

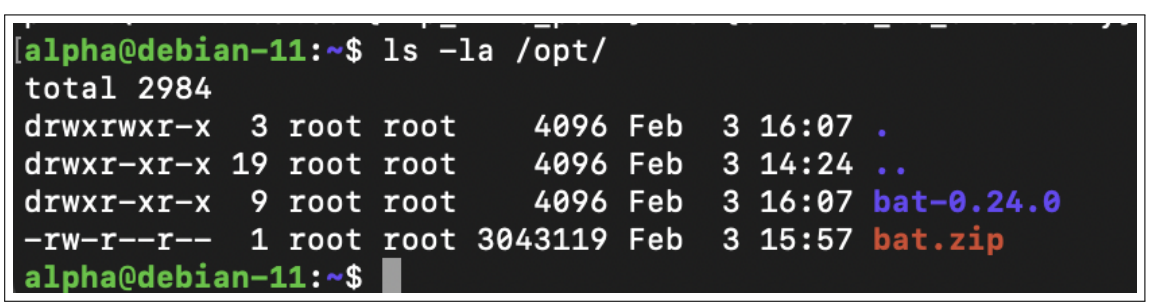


Figure 10: Successfully extracted files in /opt/

5.4 Webserver Installation

We will install Apache on the VM. First, SSH into the VM and switch to root:

```
vagrant ssh
sudo su -

```

Now, we install Apache:

```
sudo apt-get install -y apache2

```

The service fails to start, so check the logs to identify what was the error:

```
journalctl -xe

```

We encountered an issue where another process (nc) is using port 80. Hence using the command `lsof` or `ss` to identify and kill the process:

```
lsof -i :80
sudo kill <PID>

```

Finally, we verify that Apache is running by accessing the server's IP in the browser.

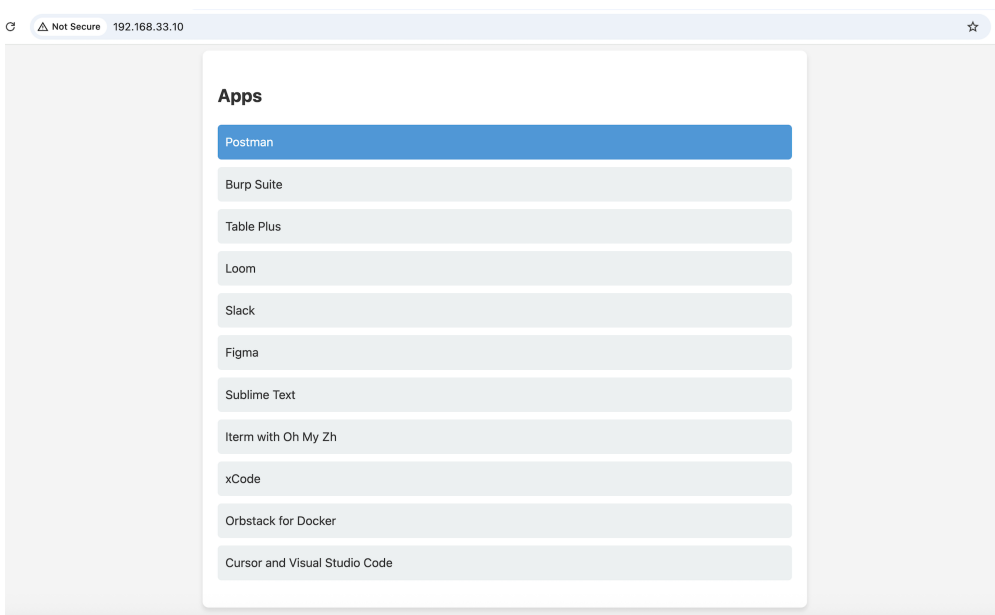


Figure 11: Apache Webserver Installation